

ОТЧЕТ

Осина Дарья, БПИ-245

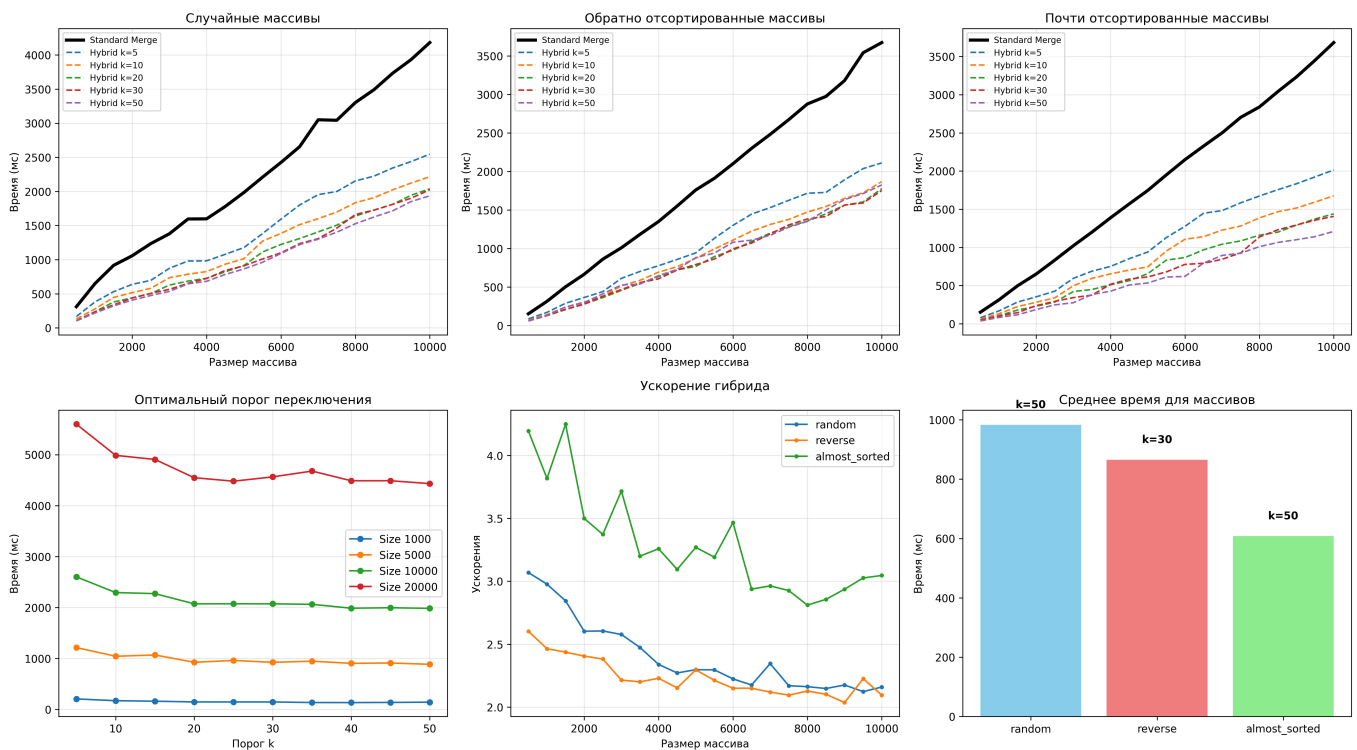
ID посылки: 349266225

Ссылка на ID:

<https://dsahse25.contest.codeforces.com/group/SLdl1pWUpC/contest/647790/submission/349266225>

Гитхаб: <https://github.com/Kriss010119/Algorithms/tree/main/SET3/A2i>

Ниже приведены графики анализа:



Можно заметить, что гибрид покруче обычной сортировки, так как работает быстрее.

Плюс, с увеличением числа элементов растет время работы алгоритма

Быстрее всего гибрид работает на почти упорядоченных массивах ~ в 3-4 раза ускорение.

На случайных и обратных данных ускоряется в 1.5-2 раза

Лучше всего переключаться на Insertion Sort, когда осталось 20-30 элементов, это поможет ускориться

Размер массива особо не влияет на порог, при котором становится выгоднее начать сортировать его простым методом а тип данных влияет сильно

Общий код

```
#include <iostream>

#include <vector>

#include <cmath>

#include <random>

#include <algorithm>

#include <chrono>

class ArrayGenerator {

private:

    std::random_device rnd;

    std::mt19937 gen;

public:

    ArrayGenerator() : gen(rnd()) {}

    std::vector<int> genRandArr(int n, int mini = 0, int maxi = 100000) {

        std::vector<int> arr(n);

        std::uniform_int_distribution<int> dist(mini, maxi);

        for (int i = 0; i < n; ++i) {

            arr[i] = dist(gen);
```

```
}
```

```
return arr;
```

```
}
```

```
std::vector<int> genRevArr(int n) {
```

```
std::vector<int> arr(n);
```

```
for (int i = 0; i < n; ++i) {
```

```
arr[i] = n - i;
```

```
}
```

```
return arr;
```

```
}
```

```
std::vector<int> genSortedArr(int n, int c = 10) {
```

```
std::vector<int> arr(n);
```

```
std::uniform_int_distribution<int> dist(0, n - 1);
```

```
for (int i = 0; i < n; ++i) {
```

```
arr[i] = i;
```

```
}
```

```
for (int i = 0; i < c; ++i) {
```

```
int i1 = dist(gen);
```

```
int i2 = dist(gen);
```

```
std::swap(arr[i1], arr[i2]);
```

```
}
```

```
return arr;
```

```
}
```

```
};
```

```
class SortTester {
```

```
private:
```

```
ArrayGenerator generator;
```

```
public:
```

```
void insertionSort(std::vector<int>& arr, int l, int r) {
```

```
for (int i = l + 1; i <= r; ++i) {
```

```
int key = arr[i];
```

```
int j = i - 1;
```

```
while (j >= l && arr[j] > key) {
```

```
arr[j + 1] = arr[j];
```

```
--j;
```

```
}
```

```
arr[j + 1] = key;
```

```
}
```

```
}
```

```
void merge(std::vector<int> &arr, int l, int m, int r) {
```

```
int n1 = m - l + 1;
```

```
int n2 = r - m;

std::vector<int> left(n1);

std::vector<int> right(n2);

for (int i = 0; i < n1; ++i) {

    left[i] = arr[l + i];

}

for (int i = 0; i < n2; ++i) {

    right[i] = arr[m + 1 + i];

}

int i = 0, j = 0, k = l;

while (i < n1 && j < n2) {

    if (left[i] <= right[j]) {

        arr[k] = left[i];

        ++i;

    } else {

        arr[k] = right[j];

        ++j;

    }

    ++k;

}

while (i < n1) {

    arr[k] = left[i];

    ++i;
```

```
++k;
```

```
}
```

```
while (j < n2) {
```

```
arr[k] = right[j];
```

```
++j;
```

```
++k;
```

```
}
```

```
}
```

```
void mergeSort(std::vector<int> &arr, int l, int r) {
```

```
if (l < r) {
```

```
int m = l + (r - l) / 2;
```

```
mergeSort(arr, l, m);
```

```
mergeSort(arr, m + 1, r);
```

```
merge(arr, l, m, r);
```

```
}
```

```
}
```

```
void hybridMergeSort(std::vector<int>& arr, int l, int r, int k) {
```

```
if (r - l + 1 <= k) {
```

```
insertionSort(arr, l, r);
```

```
return;
```

```
}
```

```
int m = l + (r - l) / 2;

hybridMergeSort(arr, l, m, k);

hybridMergeSort(arr, m + 1, r, k);

merge(arr, l, m, r);

}
```

```
long long testMergeSort(std::vector<int> arr) {

auto start = std::chrono::high_resolution_clock::now();

mergeSort(arr, 0, arr.size() - 1);

auto elapsed = std::chrono::high_resolution_clock::now() - start;

return std::chrono::duration_cast<std::chrono::microseconds>
(elapsed).count();

}
```

```
long long testHybridMergeSort(std::vector<int> arr, int k) {

auto start = std::chrono::high_resolution_clock::now();

hybridMergeSort(arr, 0, arr.size() - 1, k);

auto elapsed = std::chrono::high_resolution_clock::now() - start;

return std::chrono::duration_cast<std::chrono::microseconds>
(elapsed).count();

}

};
```

```
int main() {
```

```
SortTester sorter;

ArrayGenerator gen;


auto testData = gen.genRandArr(50000);

auto randomData = gen.genRandArr(100000);

auto reverseData = gen.genRevArr(100000);

auto almostData = gen.genSortedArr(100000);


std::vector<int> sizes;

std::vector<int> thresholds = {5, 10, 20, 30, 50};

std::vector<std::string> type_names = {"random", "reverse",
"almost_sorted"};

std::vector<std::vector<int>> test_data = {randomData, reverseData,
almostData};


for (int size = 500; size <= 10000; size += 500) {

sizes.push_back(size);

}


std::cout << "\n---> sort_res.csv\n\n";

std::cout << "type,size,algorithm,threshold,time\n";


for (int i = 0; i < 3; ++i) {

auto& data = test_data[i];
```

```

for (int size : sizes) {

if (size > data.size()) {

continue;

}

long long mergeTime = 0;

std::vector<int> arr(data.begin(), data.begin() + size);

for (int i = 0; i < 3; ++i) {

mergeTime += sorter.testMergeSort(arr);

}

std::cout << type_names[i] << "," << size << ",merge,0," << mergeTime/3 <<
"\n";

for (int k : thresholds) {

long long hybridTime = 0;

for (int i = 0; i < 3; ++i) {

hybridTime += sorter.testHybridMergeSort(arr, k);

}

std::cout << type_names[i] << "," << size << ",hybrid," << k << "," <<
hybridTime/3 << "\n";

}

}

}

std::cout << "\n---> threshold.csv\n";

std::cout << "size,threshold,time\n";

```

```
for (int size : {1000, 5000, 10000, 20000}) {  
  
    for (int k = 5; k <= 50; k += 5) {  
  
        long long time = 0;  
  
        std::vector<int> arr(testData.begin(), testData.begin() + size);  
  
        for (int i = 0; i < 3; ++i) {  
  
            time += sorter.testHybridMergeSort(arr, k);  
  
        }  
  
        std::cout << size << "," << k << "," << time/3 << "\n";  
  
    }  
  
}  
  
return 0;  
  
}
```